OPERATING GUIDE

Forge

An engineering process for AI coding sessions: research, architecture, plan, tests, code, review, evidence. Enforced by hooks, not by the model remembering to behave.

```
request --> research --> architecture --> plan --> tests --> code --> review --> evidence
                                          gate opens here              claim becomes provable
```

Installed	globally, in `~/.claude`. Every project on this machine gets the gates.
Mode	`advisory` by default. `forge strict on` per project to make gates block.
Tested	56 assertions, all passing. Hooks driven with real event JSON, not mocks.
Repos	github.com/rafiulislam4246/forge · .../continuum

Read this first: page 2 explains what was actually wrong with your setup, because the repos were never the missing piece. Pages 3 to 5 are the daily loop. Page 6 is the maximum-power section: the exact phrases that get the most out of it.

1. What was actually broken

Your setup stopped enforcing engineering on 1 November. Not gradually: on that date, in three specific ways.

FOUND ON YOUR MACHINE	WHAT IT MEANT
`~/.claude/skills/` held 6 entries; 63 skills sat in `skills-library/`	Only those 6 were auto-discoverable. The library never loads on its own.
`/stack` had `disable-model-invocation: true`	Claude **could not** call the router itself. Unless you typed `/stack`, zero skills loaded. Ever.
`skills_DELETED_2026-11-01/` held 22 skills	Including test-driven-development, planning-and-task-breakdown, code-review-and-quality, documentation-and-adrs, api-and-interface-design, debugging-and-error-recovery, incremental-implementation.
`_disabled_skills/` held 17 more	Including spec-driven-development, source-driven-development, browser-testing-with-devtools, frontend-ui-engineering.
`settings.json` → `"hooks"` absent	No deterministic enforcement of anything. Every rule depended on the model recalling it mid-task.
4 agents defined, nothing invoking them	code-reviewer, test-engineer, security-auditor, web-performance-auditor all idle.

So the complaints line up exactly. **No research, no plan, no architecture file** because those skills were deleted. **Says OK then the UI is broken** because browser-testing was disabled and nothing required evidence. **Fix one bug, get another** because no regression gate existed. **Forgets the project** because nothing re-injected context.

> **The structural lesson.** "Use TDD" in a CLAUDE.md is a suggestion. A rule only holds if something outside the conversation checks it. That is what the seven hooks do, and it is the one layer no GitHub repo ships properly.

2. What is now installed

LAYER	WHAT	WHERE
Discipline skills	16 restored, auto-triggering: TDD, planning, spec-driven, source-driven, API design, ADRs, code review, doubt-driven, debugging, browser testing, frontend, shipping, simplification, context engineering, interview-me, incremental	`~/.claude/skills/`
Forge skills	5: router, intake, build, audit, verify	`~/.claude/skills/`
Enforcement	7 hooks	`~/.claude/forge/hooks/`
CLI	`forge`, POSIX sh, no dependencies	`~/.local/bin/forge`
Router fixed	`/stack` is now model-invocable	`~/.claude/skills/stack/`
Artifacts	per project, on `forge init`	`docs/engineering/`

Your `settings.json` was backed up and merged: all 11 keys, 13 permissions and 2 MCP servers intact, with 7 hook entries added.

3. The seven hooks, and what each one catches

HOOK	FIRES ON	CATCHES
session-start	`SessionStart` incl. `compact`	A session starting blind. Injects the contract, the active plan, the current task, gate state, recent evidence. Re-fires after compaction, so a long session recovers.
prompt	`UserPromptSubmit`	A build request answered by typing code. With no plan open, the research-first rule lands next to your prompt.
pre-edit	`PreToolUse` Edit/Write	**The plan gate.** Product code before a plan. Advisory, or denied outright in strict mode.
post-bash	`PostToolUse` Bash	Unverifiable claims. Every test, build, lint and browser command is classified and logged with its result. A failure is reported back as the project's current state.
reanchor	`PostToolUse` Edit/Write	**Mid-session drift.** Every 8th edit, re-injects the current task and the out-of-scope list.
stop	`Stop`	End of turn: code changed with no passing run, or UI changed with nobody having looked at it.
task-complete	`TaskCompleted`	Completion without proof. Exit 2 blocks it in strict mode.

The three gates you can check yourself

```
forge gate code   # is there a plan with real tasks
forge gate done   # is there passing evidence newer than the last edit
forge gate ui     # did UI change without browser verification
```

Two details that make these honest rather than decorative:

- **Placeholders do not count.** A task line still containing `<angle brackets>` is scaffold. The gate stays shut until real file paths, tests and commands replace them, so it cannot be opened by generating the template.
- **The done gate compares timestamps, not claims.** Evidence recorded before the last edit proves nothing about the code as it stands now. Edit after a passing run and the gate closes again.

Never gated

Tests, docs, markdown, config, `.forge/`, `.claude/`, scripts, and anything outside your code directories. There is always a legal next move: writing the plan or the test it demands is never blocked.

> **Escape hatch, by design.** A genuinely trivial fix: `forge waive 30 "typo in the error string"`. Thirty minutes, and the bypass is written to the ledger. The point is not to prevent bypassing, it is to prevent bypassing *silently*.

4. Advisory versus strict

MODE	BEHAVIOUR	USE FOR
`advisory`	Gates inject a warning. Nothing is ever blocked.	Default. Client work, exploration, anything where an interruption costs more than a skipped step.
`strict`	Plan gate *denies* the edit. TaskCompleted blocks.	A project where quality has actually bitten you. Start with one.

Recommendation: run advisory everywhere for a week, read the ledger, then turn strict on for the one project where the warnings were right most often.

5. The four loops

Four situations, four openings. Each one is a sentence you type, not a command you memorise: the skills trigger on what you say.

A. A brand new project

```
cd ~/Work/new-thing && forge init
# then in Claude Code:
"here is the brief: <paste>. set this project up."
```

What happens: `forge-intake` interviews you, fills a 14-line completeness checklist, asks at most seven questions in one batch with recommended defaults, shows you the assumptions it would build on, and **stops**. Nothing is generated until you say OK. On your approval it writes the README, the folder structure, CLAUDE.md, .gitignore, .env.example, the test runner with one passing test, the ADRs for the stack decisions, and the first workstream's research, architecture, plan and test-plan. Then it runs `forge verify` to prove the skeleton builds.

> **This is the behaviour you asked for.** If it starts generating files before you approve, say "you have not shown me the completeness report" and it will stop. The gate is in the skill, and the skill is now auto-loading.

B. An existing project, new feature

```
cd ~/Work/review-engine && forge init    # once per project
# then:
"add <feature>. research first."
```

Research reads *your* codebase before designing, which is what stops a second implementation being built beside the one that already exists. Architecture names the rejected alternatives. The plan gate opens only when tasks carry real file paths and real verification commands.

C. An audit of something that already exists

```
forge audit
# then:
"audit this project. scope first, reproduce every finding."
```

Scope and severity are declared before any code is read, so the audit cannot drift into a tour. Eleven systematic passes: cold start, tests, correctness, error paths, auth, secrets, data, UI in a real browser, performance, observability, dependencies. Every finding needs a reproduction; anything that cannot be demonstrated becomes a note, not a finding. **It reports, it does not fix**, then turns findings into an ordered remediation plan you approve.

D. A bug that keeps coming back

```
"this is still broken: <symptom>. root cause, not symptom."
```

`debugging-and-error-recovery` loads, finds the cause, and the fix becomes a small workstream *with a test asserting the bug*. A bug fixed without that test is a bug waiting to return, which is the mechanism behind "we fixed this already".

6. Command reference

```
forge init
forge kickoff "<project>"
forge new "<title>"
forge list
forge use <slug>
forge status
forge brief
forge anchor
```

```
forge verify [test|build]
forge evidence add <kind> \
  <pass|fail> "<cmd>" "<what>"
forge evidence list
forge gate {code|done|ui}
forge waive <min> "<why>"
forge adr "<decision>"
forge audit
forge strict [on|off]
forge doctor
```

Evidence kinds: `test` `build` `lint` `browser` `api` `manual` `verify`. If `forge verify` says no commands detected, give the project a definition of working, once:

```
// .forge/config.json
{ "verify": { "test": "pnpm test", "build": "pnpm build" } }
```

7. Maximum power: what to say

The gates stop the worst outcomes on their own. Getting the *best* outcome is still about what you ask for. These are the phrases that change the result most, and why each one works.

> `"research first, then show me the architecture before you write anything"`
>
> Forces phases 1 and 2 to be visible and approvable. Most bad builds are bad decisions made silently at minute two, and this is the cheapest moment to catch one.

> `"show me the plan, I want to approve the tasks before you start"`
>
> You review 10 lines instead of 400. A wrong task is seconds to fix; wrong code is an hour.

> `"what did you actually run? paste the output"`
>
> The single highest-value sentence in this entire guide. It converts a claim into evidence, and the ledger means the answer can be checked.

> `"test counts before and after"`
>
> "Tests pass" hides a regression. Two numbers cannot.

> `"open it in a browser and click every element you touched"`
>
> Your recurring complaint, answered directly. Tests do not catch a button wired to nothing or a section that renders empty because the data shape changed.

> `"review this with a fresh subagent, do not tell it your reasoning"`
>
> The author is the worst reviewer of their own work. An unanchored reviewer finds what the author cannot see.

> `"what is still unverified?"`
>
> Surfaces the honest gap instead of a confident summary. Ask it before you believe anything is done.

> `"is this in the plan? if not, stop"`
>
> Cuts scope sprawl the moment you notice it, and amends the plan rather than letting the change drift.

Three habits that compound

1. **Read `forge status` before you start.** Five seconds. It tells you where the last session actually stopped and which gates are red, before you add anything on top.
2. **Make it tick tasks off as they pass.** The re-anchor hook reads `plan.md`. A stale plan produces a confidently wrong anchor, which is worse than none.
3. **Grow the pitfalls file.** Every time something bites, one entry. This is the part that makes month three cheaper than month one.

8. When a gate fires

GATE	RIGHT RESPONSE
Plan gate, real work	Stop and plan. The gate is working.
Plan gate, trivial fix	`forge waive 30 "<why>"`. One command, recorded.
Evidence gate	`forge verify`. If it cannot run here, say which part is unverified.
UI gate	Drive the actual page, then `forge evidence add browser pass …`.
Post-bash reports a failure	Believe the ledger over the summary. Root cause before anything else.

> **One thing to watch for.** If a session ever writes product code somewhere the hook does not look in order to get around a gate, that is a serious failure, not a clever workaround. Tell it so, and turn strict on for that project.

9. Your first week

DAY	DO THIS
Today	Install the plugins the gates expect to work beside (next section). Run `forge init` in your two or three most active projects. Nothing else.
Day 2	Run one real feature through the full pipeline, start to finish, and read every artifact it produces. This is where you decide whether the templates ask the right questions for your work.
Day 3	`forge audit` on one existing project. Expect the findings to be uncomfortable; that is the point.
Day 4	Give every active project a verify command in `.forge/config.json`. A project with none has no definition of working.
Day 5	Read the ledgers. `forge evidence list` in each project. Turn `strict on` where the warnings were right most often.

10. The plugins worth adding

Forge is the process and the enforcement. These add specialists, and they install from inside Claude Code with `/plugin`. Star counts verified against the GitHub API, not blog posts.

INSTALL	GIVES YOU
`/plugin marketplace add obra/superpowers` `/plugin install superpowers@superpowers`	315K stars. The same methodology with worktree isolation and subagent-per-task review. Your old `sp-*` skills were stripped copies of this.
`/plugin install pr-review-toolkit@claude-plugins-official`	Six review agents. One is `silent-failure-hunter`, which is your "says OK but it is broken" problem by name.
`/plugin install feature-dev@claude-plugins-official`	`code-explorer` and `code-architect` agents for the research and architecture phases.
`/plugin install playwright@claude-plugins-official`	Real browser control. This is what makes the UI gate enforceable rather than aspirational.
`npx openspec init` (per project)	74K stars. Living specs in `openspec/` if you want specs versioned separately from workstreams. Optional: Forge's artifacts already cover most of it.

> **Do not stack ECC (300K stars) on top of superpowers.** Both are full harnesses with their own hooks and its own README says to pick one path. Forge sits under either.

11. Troubleshooting

SYMPTOM	FIX
Gates never fire	`forge doctor`. Needs `.forge/` in the project and `forge-` entries in a settings.json.
`forge: command not found`	Add `~/.local/bin` to PATH, or call `~/.claude/forge/bin/forge`.
Plan gate will not open	Remove `_pending_` *and* replace the `<placeholders>` in task lines. Check with `forge gate code`.
Nudges feel too frequent	`FORGE_COOLDOWN_MIN=30`, `FORGE_ANCHOR_EVERY=15`.
Evidence ledger has a wrong entry	`forge evidence clear`, then `forge verify` for a clean baseline.
Want it gone	`./uninstall.sh --global`. Settings are stripped of forge entries only; your artifacts stay.

12. What is deliberately not here

- **No embeddings or vector search.** A handful of curated files read on demand beats retrieval that returns plausible fragments. Retrieval failure is silent; a missing file is not.
- **No background daemon.** Every state change comes from a hook or a command you ran, so the state is always explainable.
- **No blocking by default.** A process that fights you on day one gets uninstalled on day two.